

FRR OSPF-SR + DiffServ-TE

大型パケット全損失 — 障害診断レポート

作成日: 2026-06-22 実験環境: frr-docker-lab (Linux/Docker/veth)

1. 問題の概要

FRR OSPF-SR + DiffServ-TE WRR 4:2:1 ラボ実験において、iperf3 (UDP, 8950 byte) を送信したところ受信側バイト数が常に 0 となり通信が成立しなかった。小パケット (100 byte, 1450 byte) は正常に到達するが、2048 byte を超えるパケットがすべて廃棄されていた。

■ 症状一覧

テスト内容	パケットサイズ	結果
ping (小)	100 byte	✓ 0% ロス
ping (中)	1400 / 1450 byte	✓ 0% ロス
ping (大)	8900 byte	100% ロス
iperf3 UDP	8950 byte	受信 0 bytes
iperf3 control	TCP (< 1500 byte)	✓ 接続確立 (後に切断)

2. 診断プロセス

① MTU 確認 (疑い → 否定)

最初に veth ペア全区間の MTU を確認した。tx1-leri, leri-tx1, leri-cr1, cr1-leri, lere-cr1 いずれも MTU=9000 と設定済みで、MTU 不足は原因でなかった。

② PMTU キャッシュ確認 (疑い → 否定)

Tx1 のルートキャッシュ (ip route show cache) を確認・フラッシュしたが、大型 ping の失敗は続いた。

③ iptables カウンタ確認 (重要な手がかり)

LER_Ingress の iptables mangle PREROUTING DSCP MARK チェーンの ICMP

ルールカウンタを確認したところ、小パケット 10 個は計上されているが、8900 byte ping 5 個はカウンタに反映されていなかった。

→ 大型パケットは iptables に到達する前に廃棄されている、と判断。

④ tc ingress フィルタの統計確認 (原因特定)

LER_Ingress の leri-tx1 に設定されたイングレスポリサーの統計を確認:

```
tc -s filter show dev leri-tx1 parent ffff:
```

```
police 0x1 rate 42857Mbit burst 53571427b mtu 2Kb action drop overhead 0b
```

```
Sent 2359420147079 bytes 264655002 pkts (dropped 262773428, overlimits 262773428)
```

「mtu 2Kb」という値と「262,773,428 パケット廃棄」が確認された。送信総パケット数 264,655,002 のうち 262,773,428 個 (約 99.3%) が廃棄されていた。

⑤ ポリサー一時削除による確認 (根本原因の確定)

leri-tx1 のイングレス qdisc を削除 (tc qdisc del dev leri-tx1 ingress) した直後に 8900 byte ping を送信:

3 packets transmitted, 3 received, 0% packet loss rtt avg 0.120 ms

→ 大型パケットが即座に疎通。根本原因はイングレスポリサーの mtu 設定と確定。

3. 根本原因

【主因】 tc police のデフォルト mtu = 2Kb
Linux の tc police アクションは、mtu パラメータを省略するとデフォルトで 2048 byte (2 Kb) が適用される。この mtu は「正常パケット (conforming パケット) の最大サイズ」として機能し、mtu を超えるパケットはレート制限とは無関係に「overlimit」として DROP される。

問題のある設定コード (frr_dscp_te.sh 修正前):

tc filter add dev "\$dev" parent ffff: protocol all \
 u32 match u32 0 0 \
 police rate "\${rate}kbit" burst "\${burst}b" drop flowid :1

影響:

パケットサイズ	tc police の判定	結果
≤ 2048 byte	conforming (正常)	✓ 通過
> 2048 byte	overlimit (超過) として誤判定	DROP
8950 byte UDP (iperf3)	overlimit	DROP — 受信 0 bytes
8900 byte ICMP (ping -s 8900)	overlimit	DROP — 100% ロス

なぜ気づきにくかったか: バースト量 (53 MB) と転送レート (42.8 Gbps) は十分に大きく、「レート超過でドロップされている」という直感的な解釈には至りにくい。また tc の man ページに mtu のデフォルト値が明示されていないため見落としやすい。

【副因】 NETEM_LIMIT_HI = 30 (過小なキュー長)
lab_config.sh の NETEM_LIMIT_HI がデフォルト 30 パケットに設定されていた。AF41 クラスは 25G×4/7 ≈ 14.3 Gbps、8950 byte パケットで約 200 Kpps に相当する。netem キュー長 30 パケットでは数十マイクロ秒分しかバッファできず、バースト時にパケットロスが発生しやすい状態だった。

※ ただし、主因 (tc police mtu 2Kb) の修正前は大型パケットがポリサーで全廃棄されていたため、この副因の影響は顕在化していなかった。

4. 実施した修正

修正① — tc police に mtu 9000 を追加 (主修正)
対象ファイル: scripts/frr_dscp_te.sh — Ingress policing セクション

	コード
修正前	police rate "\${rate}kbit" burst "\${burst}b" drop flowid :1
修正後	police rate "\${rate}kbit" burst "\${burst}b" mtu 9000 drop flowid :1

この修正により、tc police が 9000 byte 以下のパケットを conforming として扱い、レート範囲内であれば正常に通過させるようになった。適用対象は leri-tx1/tx2/tx3 の 3 インタフェース (全 Tx→LER_Ingress 入力)。

修正② — NETEM_LIMIT_HI を 30 → 100000 に拡大 (副修正)
対象ファイル: scripts/lab_config.sh

	設定値	根拠
修正前	NETEM_LIMIT_HI=30	デフォルト値（設定根拠なし）
修正後	NETEM_LIMIT_HI=100000	25G×4/7≈200Kpps, バースト吸収に十分な余裕

5. 修正後の計測結果（2026-06-22）

■ frr_normal — WRR 4:2:1 正常動作確認

全クラスが CR1 経由で 25 Gbps を WRR 比率どおりに分配:

クラス	実測スループット	理論値 (25G×N/7)	誤差
AF41 (高優先, 重み4)	14.28 Gbps	14.29 Gbps	< 0.1%
AF42 (中優先, 重み2)	7.14 Gbps	7.14 Gbps	< 0.1%
AF43 (低優先, 重み1)	3.57 Gbps	3.57 Gbps	< 0.1%
合計	24.99 Gbps	25.00 Gbps	< 0.1%

■ frr_failure / frr_failure_reroute — 障害時自動迂回の効果

シナリオ	障害発生	通信断時間	復旧方式
frr_failure (迂回なし)	t=20s CR1 DOWN (t=40s CR1 UP)	約 21 秒 (t=21s ~ t=41s)	CR1 復旧まで全停止 (unreachable フォールバック)
frr_failure_reroute (OSPF-SR 動的迂回)	t=20s CR1 DOWN (t=40s CR1 UP)	約 1~2 秒 (t=21s のみ完全断)	frr_te_monitor が CR2 へ切替 (BFD 150ms + OSPF 収束監視)

OSPF-SR + frr_te_monitor (BFD高速検知 + operstate監視) により、CR1 障害時の通信断を 21 秒 → 1~2 秒に短縮 (約 10~20 倍の回復速度改善)。

6. まとめ

本実験で通信が成立しなかった直接原因は、tc police アクションのデフォルト mtu 値 (2048 byte = 2 Kb) によってジャンボフレーム相当のパケットが全廃棄されていたことである。この問題は iptables のカウンタが増加しないという観測から「パケットが iptables に到達する前に廃棄されている」と絞り込み、tc ingress フィルタの統計 (mtu 2Kb、廃棄 262M パケット) により確定した。

修正は frr_dscpl_te.sh の tc filter コマンドに mtu 9000 を追加するだけであったが、この 1 パラメータの省略が実験全体を無効化していた。tc のデフォルト値が man ページに明記されていないことが見落としの主因であり、ジャンボフレームを使用する環境では tc police に必ず mtu をジャンボフレーム以上に明示指定することが必要である。